

L13 · Άπληστοι Αλγόριθμοι III

Κωδικοποίηση Huffman

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Συμπίεση δεδομένων με κωδικοποίηση Huffman: απροθεματικοί κώδικες, αναπαράσταση με δυαδικά δέντρα, ο άπληστος αλγόριθμος συγχώνευσης συχνοτήτων, και η απόδειξη βελτιστότητας.

1 Τι θα δούμε

Ένα μόνο πρόβλημα — αλλά από τα ωραιότερα του μαθήματος: **πώς συμπιέζεις δεδομένα στο ελάχιστο δυνατό μήκος**; Η απάντηση, η **κωδικοποίηση Huffman**, είναι ένας καθαρός άπληστος αλγόριθμος με μια κομψή απόδειξη βελτιστότητας μέσω **επιχειρήματος ανταλλαγής** (όπως στο προηγούμενο μάθημα).

2 Το πρόβλημα

Είσοδος: μια αλφαριθμητική ακολουθία δεδομένων — π.χ. το κείμενο ενός email.

Στόχος: αναπαράστησε τα δεδομένα με τα δυφία (bits) 0, 1 στο **συντομότερο δυνατό μήκος**.

Προσοχή

Δεν είναι κρυπτογράφηση. Η Huffman είναι **τεχνική συμπίεσης** — στόχος της είναι να μικρύνει το αρχείο, όχι να το κρύψει. Η αποκωδικοποίηση πρέπει να είναι όχι μόνο εφικτή, αλλά και **μονοσήμαντη**.

Συγκεκριμένα: έχουμε n διαφορετικούς χαρακτήρες, και για καθέναν ξέρουμε τη **συχνότητα** εμφάνισής του $f_1, \dots, f_n$. Θέλουμε να αναθέσουμε σε κάθε χαρακτήρα μια **συμβολοσειρά δυφίων**, ώστε το συνολικό μήκος του κωδικοποιημένου κειμένου να είναι ελάχιστο.

3 Σταθερό vs μεταβλητό μήκος

Σταθερό μήκος: δίνουμε σε **κάθε** χαρακτήρα τον ίδιο αριθμό δυφίων. Με k δυφία κωδικοποιούνται 2^k χαρακτήρες.

- ✓ Εύκολη αποκωδικοποίηση — κόβεις ανά k δυφία.
- ✗ Δεν πετυχαίνει ελάχιστο μήκος.

Μεταβλητό μήκος: δίνουμε **λίγα** δυφία στους συχνούς χαρακτήρες και **περισσότερα** στους σπάνιους.

- ✓ Πετυχαίνει ελάχιστο μήκος.
- ✗ Πιο σύνθετη κωδικοποίηση/αποκωδικοποίηση.

Διαίσθηση

Το παράδειγμα. Αρχείο 100.000 χαρακτήρων με συχνότητες (σε χιλιάδες):

Χαρακτήρας	a	b	c	d	e	f
Συχνότητα	45	13	12	16	9	5
Σταθερό μήκος	000	001	010	011	100	101
Μεταβλητό μήκος	0	101	100	111	1101	1100

Σταθερό μήκος: $100.000 \times 3 = 300.000$ δυφία. **Μεταβλητό μήκος:** $45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4 = 224.000$ δυφία. Κέρδος **25%** — απλώς δίνοντας στο πανταχού παρόν α έναν κώδικα ενός δυφίου.

4 Το πρόβλημα της αποκωδικοποίησης

Το μεταβλητό μήκος κρύβει μια παγίδα. Έστω αλφάβητο a, b, c με κώδικες $a \rightarrow 01, b \rightarrow 010, c \rightarrow 1$. Σε ποια λέξη αντιστοιχεί η συμβολοσειρά 0101;

- $aa = 01 \sqcup 01$
- $bc = 010 \sqcup 1$

Διφορούμενο! Δύο τρόποι, η ίδια συμβολοσειρά. Το πρόβλημα: ο κώδικας του a (01) είναι **πρόθεμα** του κώδικα του b (010).

Κλειδί

Απροθεματική κωδικοποίηση (prefix-free code): μια συνάρτηση c που αντιστοιχεί κάθε χαρακτήρα σε μια συμβολοσειρά δυφίων, έτσι ώστε για κάθε δύο διαφορετικούς χαρακτήρες $x \neq y$, η $c(x)$ να μην είναι πρόθεμα της $c(y)$.

Με απροθεματικό κώδικα η αποκωδικοποίηση γίνεται **μονοσήμαντη**. Π.χ. με $c(a) = 11, c(b) = 01, c(d) = 001, c(e) = 10, c(f) = 000$, η συμβολοσειρά 11010001010001 διαβάζεται μοναδικά ως abfeed.

5 Κόστος μιας κωδικοποίησης

Κλειδί

Το **κόστος** μιας κωδικοποίησης c είναι ο συνολικός αριθμός δυφίων:

$$\text{cost}(c) = \sum_{x \in S} f_x \cdot |c(x)|$$

όπου $|c(x)|$ το μήκος του κώδικα του x . Ζητάμε **απροθεματική** κωδικοποίηση που **ελαχιστοποιεί** αυτό το άθροισμα.

6 Κωδικοποιήσεις ως δυαδικά δέντρα

Κάθε απροθεματική κωδικοποίηση αντιστοιχεί σε ένα **δυαδικό δέντρο**:

- Οι **χαρακτήρες** κάθονται στα **φύλλα**.
- Το μονοπάτι από τη ρίζα στο φύλλο δίνει τον κώδικα: **0** = αριστερό παιδί, **1** = δεξιό παιδί.

Διαίσθηση

Γιατί αυτό εγγυάται απροθεματικότητα; Αν ένας χαρακτήρας είναι σε **φύλλο**, κανένας άλλος χαρακτήρας δεν βρίσκεται «κάτω» από αυτόν. Άρα κανένας κώδικας δεν είναι πρόθεμα κάποιου άλλου — εξ ορισμού του φύλλου. **Αποκωδικοποίηση:** διάβαζε δυφία ακολουθώντας το μονοπάτι από τη ρίζα· μόλις φτάσεις σε φύλλο, βρήκες έναν χαρακτήρα — ξεκίνα ξανά από τη ρίζα.

Σε αυτή τη γλώσσα, το $|c(x)|$ είναι ακριβώς το **βάθος** $d_T(\ell_x)$ του φύλλου του x , και θέλουμε δέντρο που ελαχιστοποιεί το $\sum_x f_x \cdot d_T(\ell_x)$.

7 Η άπληστη ιδέα

Κλειδί

Άπληστο κριτήριο. Χαρακτήρες με **μικρές** συχνότητες πάνε σε φύλλα **μακριά** από τη ρίζα (μεγάλος κώδικας)· χαρακτήρες με **μεγάλες** συχνότητες πάνε σε φύλλα **κοντά** στη ρίζα (μικρός κώδικας).

Πώς το χτίζουμε; **Από κάτω προς τα πάνω.** Παίρνουμε τους δύο **σπανιότερους** χαρακτήρες και τους κάνουμε αδέρφια — αυτοί θα είναι οι πιο βαθιά. Τους «συγχωνεύουμε» σε έναν εικονικό χαρακτήρα με συχνότητα το άθροισμά τους, και επαναλαμβάνουμε.

```
Huffman(S): // S = χαρακτήρες με συχνότητες
  Εάν |S| = 2:
    επίστρεψε δέντρο με ρίζα και δύο φύλλα
  Αλλιώς:
    έστω y, z οι δύο χαρακτήρες με τη μικρότερη συχνότητα στο S
    S' <- S
    αφαίρεσε τα y, z από το S'
    εισήγαγε νέο χαρακτήρα ω στο S' με f_ω = f_y + f_z
    T' <- Huffman(S')
    T <- το T' με την ω να αποκτά δύο παιδιά, τα φύλλα y και z
    επίστρεψε T
```

Διαίσθηση

Διαισθητικά: οι δύο σπανιότεροι χαρακτήρες «αξίζουν» τις χειρότερες θέσεις. Τους κλειδώνουμε αμέσως ως αδέρφια στο βαθύτερο σημείο, τους αντιμετωπίζουμε από εκεί και πέρα σαν **ένα** σύνθετο σύμβολο, και λύνουμε το ίδιο πρόβλημα με έναν χαρακτήρα λιγότερο.

7.1 Παράδειγμα

Χαρακτήρες με συχνότητες $a:45, e:12, f:13, g:16, b:5, d:9$. Συγχωνεύσεις, κάθε φορά οι δύο μικρότερες: $b + d = 14, e + f = 25, 14 + g = 30, 25 + 30 = 55, a + 55 = 100$. Το δέντρο που προκύπτει δίνει τους

κώδικες $a = 0, e = 100, f = 101, g = 111, b = 1100, d = 1101$: συχνά κοντά στη ρίζα, σπάνια βαθιά.

7.2 Πολυπλοκότητα

Ο αλγόριθμος κάνει $n - 1$ συγχωνεύσεις. Κάθε συγχώνευση χρειάζεται να βρει τις δύο μικρότερες συχνότητες — με μια **συντάξη προτεραιότητας** (από το μάθημα των δομών δεδομένων) κοστίζει $O(\log n)$. Άρα

$$T(n) = T(n - 1) + O(\log n) \implies T(n) = O(n \log n).$$

8 Ορθότητα

Η απόδειξη στηρίζεται σε ένα **επιχείρημα ανταλλαγής** φύλλων.

8.1 Λήμμα 1 — βάθος αντιστρόφως ανάλογο της συχνότητας

Κλειδί

Λήμμα. Σε ένα βέλτιστο δέντρο T , αν δύο χαρακτήρες έχουν συχνότητες $f(k) \leq f(j)$, τότε $d_T(\ell_k) \geq d_T(\ell_j)$ — ο σπανιότερος είναι **τουλάχιστον τόσο βαθιά**.

Απόδειξη. Έστω, για άτοπο, $f(k) < f(j)$ αλλά $d_T(\ell_k) < d_T(\ell_j)$. **Ανταλλάσσουμε** τα φύλλα των k και j , παίρνοντας κωδικοποίηση C' . Η διαφορά κόστους:

$$\begin{aligned} L(C) - L(C') &= f(k) d_T(\ell_k) + f(j) d_T(\ell_j) - f(k) d_T(\ell_j) - f(j) d_T(\ell_k) \\ &= (f(k) - f(j)) (d_T(\ell_k) - d_T(\ell_j)) > 0 \end{aligned}$$

(γινόμενο δύο **αρνητικών** ποσοτήτων). Άρα $L(C') < L(C)$ — το T **δεν ήταν** βέλτιστο. **Άτοπο. ■**

8.2 Λήμμα 2 — δομή του βέλτιστου δέντρου

Κλειδί

Σε κάθε βέλτιστο δέντρο, κάθε **εσωτερική** κορυφή έχει **ακριβώς 2 παιδιά**. Επιπλέον, οι δύο **μεγαλύτερου μήκους** κώδικες ανήκουν στους **δύο σπανιότερους** χαρακτήρες και διαφέρουν **μόνο στο τελευταίο δυφίο** (είναι αδέλφια).

Αν μια κορυφή v είχε **ένα μόνο** παιδί u , θα μπορούσαμε να «συνθλίψουμε» την ακμή $\{v, u\}$ — μικραίνοντας κάθε κώδικα από κάτω και άρα το κόστος. Άρα στο βέλτιστο δέντρο αυτό δεν συμβαίνει. Από το Λήμμα 1, οι δύο σπανιότεροι χαρακτήρες είναι οι βαθύτεροι — και μπορούμε να τους θεωρήσουμε **αδέλφια**.

8.3 Θεώρημα — η Huffman είναι βέλτιστη

Κλειδί

Θεώρημα. Για κάθε πλήθος χαρακτήρων n και κάθε συνάρτηση συχνοτήτων, το δέντρο που παράγει ο αλγόριθμος Huffman αποτελεί **βέλτιστη** κωδικοποίηση.

Απόδειξη (επαγωγή στο $|S|$).

- *Βάση $n = 2$* : το δέντρο με μία ρίζα και δύο φύλλα είναι πάντα βέλτιστο.
- *Επαγωγική υπόθεση*: το θεώρημα ισχύει για $n - 1$ χαρακτήρες.
- *Επαγωγικό βήμα*: έστω, για άτοπο, ότι το δέντρο T του αλγορίθμου για n χαρακτήρες **δεν** είναι βέλτιστο. Τότε υπάρχει δέντρο Z με $\text{cost}(Z) < \text{cost}(T)$ — και (Λήμμα 2) μπορούμε να το πάρουμε με τους δύο σπανιότερους χαρακτήρες x, y **αδέρφια**.

Από το Z φτιάχνουμε το Z' : αφαιρούμε τα x, y και βάζουμε στη θέση του κοινού γονέα τους έναν χαρακτήρα ω με $f_\omega = f_x + f_y$. Όμοια, από το T φτιάχνουμε το T' . Κρίσιμη ταυτότητα:

$$\text{cost}(Z') = \text{cost}(Z) - f_\omega \quad \text{cost}(T') = \text{cost}(T) - f_\omega$$

(όταν «βγάξεις» τα δύο αδέλφια, χάνεις ακριβώς $f_x + f_y$ από το κόστος — κάθε ένα έχανε ένα επίπεδο βάθους). Αφού $\text{cost}(Z) < \text{cost}(T)$, αφαιρώντας το ίδιο f_ω : $\text{cost}(Z') < \text{cost}(T')$.

Αλλά το T' είναι ακριβώς το δέντρο που παράγει ο αλγόριθμος για $n - 1$ χαρακτήρες — και από την **επαγωγική υπόθεση** είναι βέλτιστο. Δεν μπορεί να υπάρχει Z' φθηνότερο. **Άτοπο. ■**

Διαίσθηση

Η ψυχή της απόδειξης: η άπληστη πρώτη κίνηση — «κάνε τους δύο σπανιότερους αδέλφια» — δεν θυσιάζει τίποτα, γιατί κάποιο βέλτιστο δέντρο **ούτως ή άλλως** τους έχει αδέλφια (Λήμμα 2). Άρα μπορούμε με ασφάλεια να τους κλειδώσουμε και να αναχθούμε στο ίδιο πρόβλημα με έναν χαρακτήρα λιγότερο.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. **Κωδικοποίηση Huffman** — συμπίεση δεδομένων: λίγα δυφία στους συχνούς χαρακτήρες, πολλά στους σπάνιους.
2. **Απροθεματικός κώδικας** — κανένας κώδικας δεν είναι πρόθεμα κάποιου άλλου· εγγυάται μονοσήμαντη αποκωδικοποίηση.
3. Κάθε απροθεματικός κώδικας = **δυναμικό δέντρο** με χαρακτήρες στα φύλλα· κόστος $\sum_x f_x \cdot d_T(\ell_x)$.
4. **Άπληστος αλγόριθμος**: συγχώνευσε επανειλημμένα τους **δύο σπανιότερους** χαρακτήρες. $O(n \log n)$ με ουρά προτεραιότητας.
5. **Ορθότητα** μέσω επιχειρήματος ανταλλαγής φύλλων + επαγωγής: η άπληστη πρώτη κίνηση συμφωνεί με κάποιο βέλτιστο δέντρο, και η ταυτότητα $\text{cost}(T') = \text{cost}(T) - f_\omega$ ανάγει το πρόβλημα σε μικρότερο.